

CSES 1 : Sorting and Searching

4. Concert Tickets

Time limit: 1.00 s **Memory limit:** 512 MB

There are n concert tickets available, each with a certain price. Then, m customers arrive, one after another.

Each customer announces the maximum price they are willing to pay for a ticket, and after this, they will get a ticket with the nearest possible price such that it does not exceed the maximum price.

Input

The first input line contains integers n and m : the number of tickets and the number of customers.

The next line contains n integers $h_1, h_2, \dots, h_n$: the price of each ticket.

The last line contains m integers $t_1, t_2, \dots, t_m$: the maximum price for each customer in the order they arrive.

Output

Print, for each customer, the price that they will pay for their ticket. After this, the ticket cannot be purchased again.

If a customer cannot get any ticket, print -1 .

Constraints

- $1 \leq n, m \leq 2 \cdot 10^5$
- $1 \leq h_i, t_i \leq 10^9$

Example

Input:

```
5 3
5 3 7 8 5
4 8 3
```

Output:

```
3
8
-1
```

- need a multiset to store ticket prices since two tickets can have same prices
- then as each customer arrives, use lower_bound on it, if val is found, remove ticket and return price
 - else, out -1 since nothing is found and all prices > max willing price of current customer
- [3,5,5,7,8] , [4,8,3]
- just need to find an element in the set which is closest but not exceeding max_price.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      ios::sync_with_stdio(0);
7      cin.tie(0);
8      int n,m;
9      cin >> n >> m;
10
11     multiset <int> s;
12     for (int i=0; i<n; i++){
13         int x;
14         cin >> x;
15         s.insert(x);
16     }
17
18     int max_price;
19     for (int i=0; i<m; i++){
20         cin >> max_price;
21         auto upper_it = s.upper_bound(max_price);
22
23         if (upper_it == s.begin()){
24             // all elements strictly greater
25             cout << -1 << " ";
26             continue;
27         }
28         else if (upper_it == s.end()){
29             // all elements equal or lesser
30             auto last = s.end();
31             last--;
32             cout << *last << " ";
33             s.erase(last);
34             continue;
35         }
36
37         upper_it--;
38         cout << *upper_it << " ";
39         s.erase(upper_it);
40     }
41
42     return 0;
43 }

```

5. Restaurant Customers

Time limit: 1.00 s **Memory limit:** 512 MB

You are given the arrival and leaving times of n customers in a restaurant.

What was the maximum number of customers in the restaurant at any time?

Input

The first input line has an integer n : the number of customers.

After this, there are n lines that describe the customers. Each line has two integers a and b : the arrival and leaving times of a customer.

You may assume that all arrival and leaving times are distinct.

Output

Print one integer: the maximum number of customers.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a < b \leq 10^9$

Example

Input:

3

5 8

2 4

3 9

Output:

2

- naive: for each time step (1 to 10^9), compare with arrival and leaving times of each customer ($2 \cdot 10^5$ comparisons).
 - compute the current max overlap of customers at current time step.

<https://www.geeksforgeeks.org/dsa/maximum-number-of-overlapping-intervals/>

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      int n;
7      cin >> n;
8      vector<pair<int,char>> p;
9      for (int i=0; i<n; i++){
10         int x,y;
11         cin >> x >> y;
12         p.push_back({x,'X'});
13         p.push_back({y,'Y'});
14     }
15
16     sort(p.begin(), p.end());
17
18     int count = 0;
19     int ans = 0;
20     for (int i=0; i<2*n; i++){
21         pair<int,char> t = p[i];
22         if (t.second == 'X'){
23             count++;
24         }
25         else{
26             count--;
27         }
28         ans = max(ans, count);
29     }
30     cout << ans;
31     return 0;
32 }
```

6. Movie Festival

Time limit: 1.00 s Memory limit: 512 MB

In a movie festival n movies will be shown. You know the starting and ending time of each movie. What is the maximum number of movies you can watch entirely?

Input

The first input line has an integer n : the number of movies.

After this, there are n lines that describe the movies. Each line has two integers a and b : the starting and ending times of a movie.

Output

Print one integer: the maximum number of movies.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a < b \leq 10^9$

Example

Input:

```
3
3 5
4 9
5 8
```

Output:

```
2
```

- chapter 6, scheduling section. greedy approach
- algo that **chooses that event which ends the earliest** always produces the optimal solution.
 - im not really able to fully hear the music on this one

7. Sum of Two Values (2sum)

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an array of n integers, and your task is to find two values (at distinct positions) whose sum is x .

Input

The first input line has two integers n and x : the array size and the target sum.

The second line has n integers $a_1, a_2, \dots, a_n$: the array values.

Output

Print two integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x, a_i \leq 10^9$

Example

Input:

```
4 8
2 7 5 1
```

Output:

```
2 4
```

- traverse the array
- create dict {target-a_i : i}
- at each element, search for key a_i in dict, return value + current i if found, else add to dict and move on

8. Maximum Subarray Sum

Time limit: 1.00 s Memory limit: 512 MB

Given an array of n integers, your task is to find the maximum sum of values in a contiguous, nonempty subarray.

Input

The first input line has an integer n : the size of the array.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Output

Print one integer: the maximum subarray sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq x_i \leq 10^9$

Example

Input:

8
-1 3 -2 5 3 -5 2 2

Output:

9

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main (int argc, char *argv[]) {
6      ios::sync_with_stdio(0);
7      cin.tie(0);
8
9      int n;
10     cin >> n;
11     vector<int> a (n,0);
12     for (int i=0; i<n; i++){
13         cin >> a[i];
14     }
15
16     // find maximum sum of a sub array ending at index k
17     // either itself or itself + max sub arr sum ending at index k-1
18     long long int max_sum = a[0];
19     long long int prev = a[0];
20
21     for (int i=1; i<n; i++){
22         prev = a[i] > prev + a[i] ? a[i] : prev + a[i];
23         max_sum = max_sum > prev ? max_sum : prev;
24     }
25
26     cout << max_sum;
27
28     return 0;
29 }
```

- had difficulty in cleaning it up
- book ch1/2

- algo 1: window starting ptr loop, window ending ptr loop, within window accumulate sum loop, global tracking max $O(n^3)$
 - algo 2: remove accumulate loop, can accumulate same time as advancing ending window ptr loop
 - algo 3: find the maximum subarr sum of a sub arr ending at index k
 - this is either the element $arr[k]$, or $dp[k-1] + arr[k]$
 - final ans is max of this dp arr
-

9. Stick Lengths

Time limit: 1.00 s **Memory limit:** 512 MB

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length.

You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length.

What is the minimum total cost?

Input

The first input line contains an integer n : the number of sticks.

Then there are n integers: $p_1, p_2, \dots, p_n$: the lengths of the sticks.

Output

Print one integer: the minimum total cost.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$

Example

Input:

```
5
2 3 1 5 2
```

Output:

```
5
```

- let target length of all be x
 - this is screaming minimizing sums of $|x-l_1| + |x-l_2| + \dots + |x-l_n|$
 - $x = \text{median of all the lengths}$
 - floor of total by 2
 - sort and output, $n \log n$ will work here, 1s
-

10. Missing Coin Sum

Time limit: 1.00 s Memory limit: 512 MB

You have n coins with positive integer values. What is the smallest sum you cannot create using a subset of the coins?

Input

The first line has an integer n : the number of coins.

The second line has n integers $x_1, x_2, \dots, x_n$: the value of each coin.

Output

Print one integer: the smallest coin sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Example

Input:

5
2 9 1 2 7

Output:

6